

```
# User-defined exception
```

```
class BankingError(Exception):
```

```
    pass
```

```
class BankAccount:
```

```
    def __init__(self, acc_id, name, balance=0.0, is_active=True):
```

```
        self.acc_id = acc_id
```

```
        self.name = name
```

```
        self.balance = balance
```

```
        self.is_active = is_active
```

```
        self.daily_limit = 5000.0
```

```
        self.spent_today = 0.0
```

```
    def deposit(self, amount):
```

```
        if not self.is_active:
```

```
            raise BankingError("Account is inactive. Cannot deposit.")
```

```
        # Assertion for negative amount
```

```
        assert amount >= 0, "Deposit amount cannot be negative."
```

```
        self.balance += amount
```

```
        print(f"Deposited ${amount}. New Balance: ${self.balance}")
```

```
    def withdraw(self, amount):
```

```
        # Assertion for negative amount
```

```
        assert amount >= 0, "Withdrawal amount cannot be negative."
```

```
        if amount > self.balance:
```

```
            raise BankingError("Insufficient balance.")
```

```

    if (self.spent_today + amount) > self.daily_limit:

        raise BankingError("Daily withdrawal limit exceeded.")

    self.balance -= amount

    self.spent_today += amount

    print(f"Withdrew ${amount}. New Balance: ${self.balance}")

def transfer(self, target, amount):

    if target is None or not target.is_active:

        raise BankingError("Target account is missing or inactive.")

    # Assertion for negative amount

    assert amount >= 0, "Transfer amount cannot be negative."

    if amount > self.balance:

        raise BankingError("Insufficient balance for transfer.")

    self.balance -= amount

    target.balance += amount

    print(f"Transferred ${amount} to {target.name}. Your Balance: ${self.balance}")

# --- User Input Interaction ---

# Setup sample accounts

accounts = {

    "101": BankAccount("101", "Alice", 1000.0),

    "102": BankAccount("102", "Bob", 500.0),

    "103": BankAccount("103", "Charlie", 0.0, is_active=False)

}

```

```
print("Welcome to User-Defined Banking System")

user_id = input("Enter your Account ID (101/102): ")

my_acc = accounts.get(user_id)


if my_acc:

    try:

        print("\n1. Deposit\n2. Withdraw\n3. Transfer")

        choice = input("Select Operation: ")


        if choice == '1':

            amt = float(input("Enter deposit amount: "))

            my_acc.deposit(amt)


        elif choice == '2':

            amt = float(input("Enter withdrawal amount: "))

            my_acc.withdraw(amt)


        elif choice == '3':

            dest_id = input("Enter destination Account ID: ")

            dest_acc = accounts.get(dest_id)

            amt = float(input("Enter transfer amount: "))

            my_acc.transfer(dest_acc, amt)

        else:

            print("Invalid Choice")


    except BankingError as e:
```

```
        print(f"BANKING ERROR: {e}")

    except AssertionError as e:

        print(f"ASSERTION ERROR: {e}")

    except ValueError:

        print("INPUT ERROR: Please enter numeric values for amounts.")

else:

    print("Account not found.")
```

OUTPUT:

```
Output
Welcome to User-Defined Banking System
Enter your Account ID (101/102): 102

1. Deposit
2. Withdraw
3. Transfer
Select Operation: 2
Enter withdrawal amount: 200
Withdrew $200.0. New Balance: $300.0

=== Code Execution Successful ===
```

